



PROYECTO PRIMER PARCIAL

CDDEIA-ELNO-6-2

Ciencias de Datos e inteligencia Artificial

Redes Neuronales y Aprendizaje Profundo

Modelado de Lenguaje con RNN Elman

Grupo 1

Alvarez Roberto

Carlos Jeancarlos

Intriago Jorge

Menoscal Julleysi

Remache Steven

Paper asignado:

Revisiting Simple Neural Probabilistic Language Models

Sun & Iyyer, ACL 2021 · [arXiv:2104.03474](https://arxiv.org/abs/2104.03474)

Declaración de uso de IA:

Partes de este informe y del notebook fueron desarrolladas con asistencia de herramientas de IA.

1 — Resumen del Paper

1.1 Problema abordado

El paper de Sun & Iyyer (2021) revisita el Neural Probabilistic Language Model (NPLM) de Bengio et al. (2003), la arquitectura neuronal más simple propuesta para el modelado de lenguaje. El problema central es predecir la siguiente palabra dado un contexto de palabras anteriores, es decir, estimar la probabilidad condicional $P(w_t | w_{<t})$.

La pregunta de investigación es: ¿En qué medida las limitaciones del NPLM original se mitigan al aplicarle técnicas modernas de diseño de redes neuronales, hardware y optimización?

1.2 Metodología

Los autores diseñaron un NPLM modernizado que incorpora las siguientes mejoras sobre el modelo original de Bengio et al. (2003):

- Mayor profundidad: de 1 capa a 16 capas feed-forward, con dimensiones de embedding (410) y capa oculta (2100) sustancialmente mayores.
- Mejoras de optimización: conexiones residuales (He et al., 2016), normalización de capa (Ba et al., 2016), dropout, activación ReLU y optimizador Adam con scheduler coseno.
- Ventana de contexto ampliada: hasta $k = 50$ tokens previos (originalmente $k = 5$).
- Representación global del contexto: una convolución 1-D aprendida sobre los embeddings fuera de la ventana local.
- Softmax adaptativo y pesos atados para reducir el tamaño del modelo y acelerar el entrenamiento.

La función de composición del NPLM original (Ecuación 1):

$$z = \tanh(W [x_{t-k-1} ; x_{t-k} ; \dots ; x_{t-1}])$$

La probabilidad de la siguiente palabra se calcula como:

$$P(w_t | w_{<t}) = \text{softmax}(W_o * z + b)$$

1.3 Métrica: Perplexity y su relación con la Entropía Cruzada

La perplexity (PPL) es la métrica estándar para evaluar modelos de lenguaje. Mide cuán bien el modelo predice el corpus de evaluación: un PPL de X significa que el modelo es tan incierto como si eligiera uniformemente entre X palabras en cada paso. Su definición matemática es:

$$PPL = \exp(H(p, q)) = \exp(-1/N * \text{SUMA } \log P(w_i | w_{<i}))$$

donde $H(p, q)$ es la entropía cruzada entre la distribución real p y la predicha por el modelo q . En la práctica, con PyTorch:

$$PPL = \exp(L_{CE})$$

donde L_{CE} es la función de pérdida CrossEntropyLoss. Minimizar la entropía cruzada es equivalente a minimizar la perplexity. Un modelo perfecto tendría $PPL = 1$ (certeza absoluta en cada predicción).

1.4 Contribución principal y resultados

El NPLM modernizado pasa de PPL = 216 a PPL = 31.7 en WIKITEXT-103, reducción del 85%, sólo aplicando técnicas generales modernas. Adicionalmente, el NPLM supera al Transformer cuando el contexto de entrada es corto (< 20 tokens), lo que inspira dos variantes híbridas (Transformer-N y Transformer-C) que mejoran consistentemente al Transformer base.

Tabla 1: Resultados reportados en el paper (WikiText-2)

Modelo	Val PPL	Test PPL	Parámetros
NPLM (paper)	120.5	114.3	13M
Transformer (paper)	117.6	111.1	13M
Transformer-C	113.1	107.5	13M
Transformer-N	110.8	105.6	13M

Tabla 1: Resultados en WikiText-2 según Sun & Iyyer (2021).

2 — Dataset: WikiText-2

2.1 Descripción del dataset

WikiText-2 es un corpus de modelado de lenguaje a nivel de palabra extraído de artículos de Wikipedia de calidad verificada. Es el benchmark estándar para comparar modelos de lenguaje de tamaño moderado. Sus características principales son:

- Aproximadamente 2 millones de tokens en entrenamiento.
- Vocabulario de ~66,059 palabras únicas.
- Tres splits: train (~2M tokens), validation (~217K), test (~245K).
- Texto en inglés, con signos de puntuación preservados y mayúsculas en nombres propios.

2.2 Carga del Dataset

El dataset se cargó mediante `torchtext.datasets.WikiText2` con un parche de compatibilidad MD5, necesario porque el servidor original de descarga no es confiable en entornos como Colab. El dataset se descarga desde HuggingFace en formato `.parquet`, se convierte al formato `.tokens` esperado por `torchtext`, y se empaqueta en un `.zip` con el MD5 corregido. No se incluye el archivo de datos en el repositorio por exceder el límite de 50 MB.

2.3 Análisis Exploratorio (EDA)

Tabla 2: Estadísticos descriptivos de WikiText-2

Split	Tokens	% del total
Train	~2,075,432	89.1%
Validation	~217,126	9.3%
Test	~245,047	10.5%

Tabla 2: Distribución de tokens por split en WikiText-2.

La Figura 1 muestra las 30 palabras más frecuentes del corpus y la distribución de frecuencias en escala log-log. Se observa una clara ley de Zipf: pocas palabras (artículos, preposiciones) concentran la mayoría de las ocurrencias, mientras que la larga cola del vocabulario contiene palabras que aparecen una sola vez (hapáx legómena).

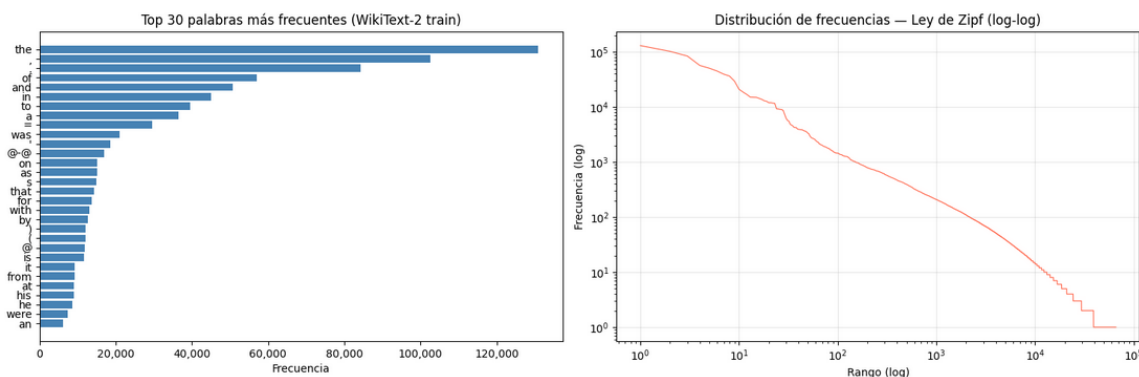


Figura 1: (a) Top 30 palabras más frecuentes. (b) Distribución de frecuencias en log-log (Ley de Zipf). Generada en el notebook.

La Figura 2 muestra la distribución de longitud de líneas en el corpus de entrenamiento, con una media de aproximadamente 86.3 tokens por línea.

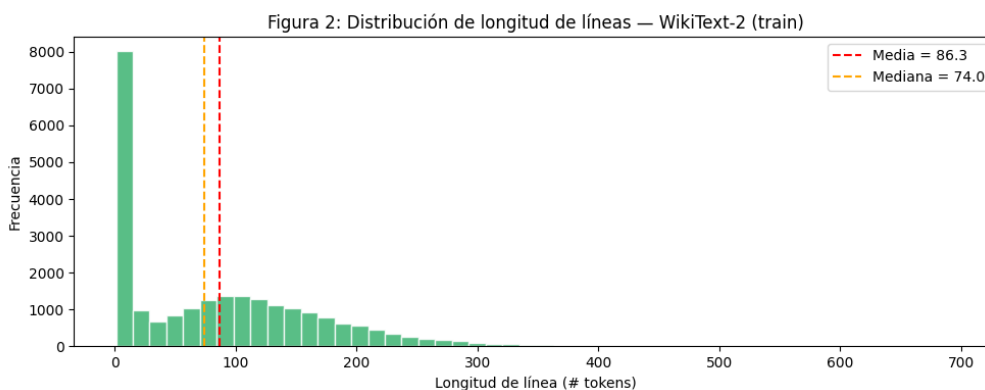


Figura 2: Distribución de longitud de líneas (número de tokens) en WikiText-2 train.

La Figura 3 desglosa el vocabulario por rangos de frecuencia: más del 50% de las palabras únicas aparecen solo una vez, lo que justifica el uso de un token especial <unk>.

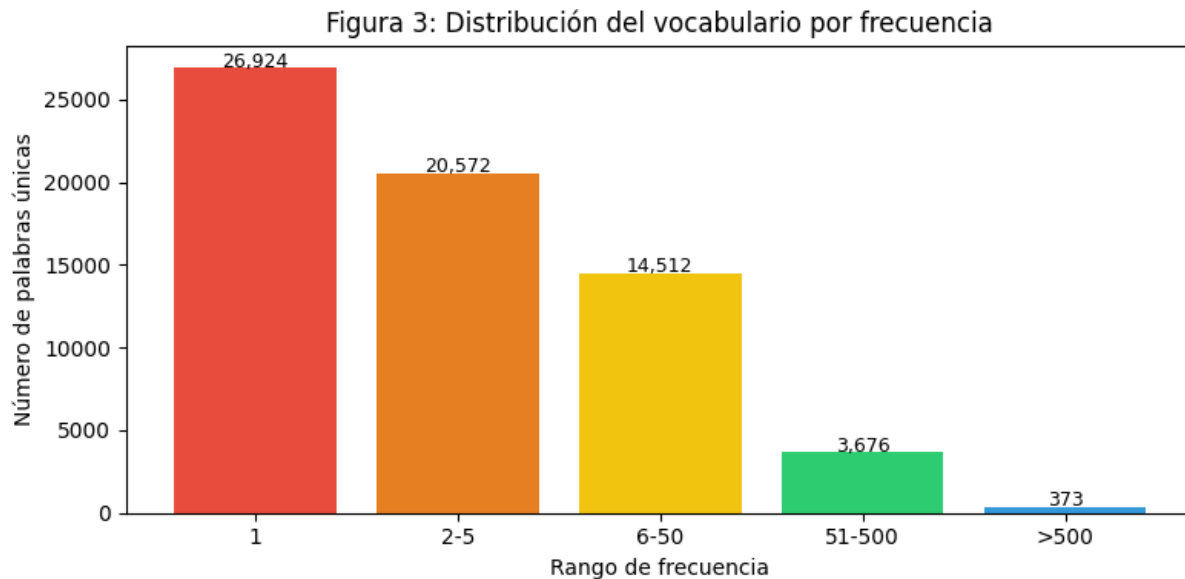


Figura 3: Distribución del vocabulario por rango de frecuencia.

3 — Implementación: RNN Elman desde cero

3.1 Arquitectura del modelo

Se implementó una RNN Elman (red recurrente simple) para la tarea de predicción de la siguiente palabra. La arquitectura, implementada completamente desde cero sin usar nn.RNN de PyTorch como caja negra, consta de las siguientes capas:

1. Embedding layer: convierte índices de tokens en vectores densos de dimensión $d_{emb} = 512$, compartida con la capa de salida mediante weight tying.
2. Dos celdas ElmanRNN apiladas (implementación manual): cada una calcula el estado oculto con la ecuación característica, con dropout entre capas.
3. Dropout ($p = 0.4$): aplicado entre capas para regularización.
4. Capa lineal de salida (decoder): comparte pesos con el embedding (weight tying), reduciendo parámetros y mejorando la generalización.

La ecuación de la celda Elman es:

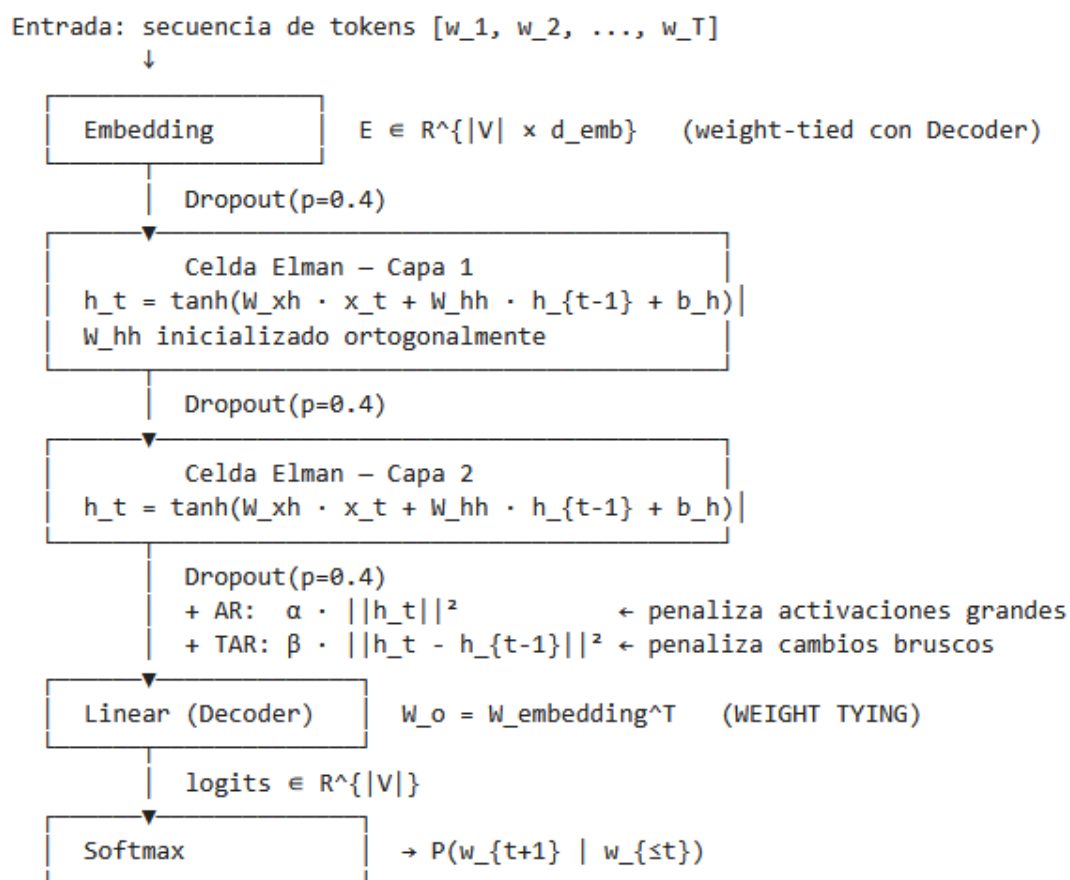
$$h_t = \tanh(x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h)$$

donde x_t es el embedding del token en el instante t , h_{t-1} es el estado oculto anterior, W_{xh} in $\mathbb{R}^{d_{emb} \times d_{hidden}}$ y W_{hh} in $\mathbb{R}^{d_{hidden} \times d_{hidden}}$ son matrices de pesos aprendibles.

3.2 Inicialización de pesos

Se usó inicialización de Kaiming para W_{xh} e inicialización ortogonal para W_{hh} . La inicialización ortogonal es especialmente importante para W_{hh} porque produce una matriz cuyas columnas son ortonormales, lo que preserva mejor la norma del gradiente durante el backpropagation a través del tiempo y mitiga parcialmente el problema del gradiente que desaparece.

3.3 Diagrama del modelo



Optimizador: Adam $lr=3e-4$, $weight_decay=1e-5 \rightarrow$ LR decay (factor=0.5, patience=5)
 Gradient clipping: 0.25

Figura 4: Diagrama de la arquitectura RNN Elman implementada.

3.4 Hiperparámetros

Hiperparámetro	Valor
Dimensión embedding (d_emb)	512
Tamaño estado oculto (d_hidden)	512
Dropout	0.4
Long. de secuencia BPTT (seq_len)	35
Tamaño de batch	20
Optimizador	Adam (lr=3e-4, weight_decay=1e-5)
Gradient clipping	0.25
Épocas máximas	30
Scheduler	ReduceLROnPlateau (factor=0.5, patience=5)
Capas Elman (N_LAYERS)	2
Weight Tying	Activado (emb ↔ decoder)
AR (Activation Reg.) α	0.3
TAR (Temporal AR) β	0.15

Tabla 3: Hiperparámetros del modelo RNN Elman optimizado.

4 — Entrenamiento, Evaluación y Resultados

4.1 Entrenamiento con BPTT

El entrenamiento se realizó mediante Backpropagation Through Time (BPTT) truncado con longitud de secuencia `seq_len = 35`. En BPTT, los datos de entrenamiento se organizan en columnas de tamaño `batch_size` y se procesan en segmentos de longitud `seq_len`, desplazando los targets un paso hacia adelante respecto a los inputs. El estado oculto se pasa entre segmentos pero se desconecta del grafo computacional (`.detach()`) para limitar la profundidad del backpropagation.

Se aplicó gradient clipping con un umbral de 0.25 para prevenir la explosión de gradientes, problema común en RNNs simples con secuencias largas. El umbral limita la norma global del gradiente: si la norma supera el umbral, todos los gradientes se escalan proporcionalmente.

4.2 Curvas de entrenamiento

La Figura 4 muestra las curvas de pérdida (cross-entropy loss) en entrenamiento y validación a lo largo de las épocas. La Figura 5 muestra la perplexity en validación. Se guarda automáticamente el modelo con la menor pérdida de validación, que es el que se evalúa finalmente en el conjunto de test.

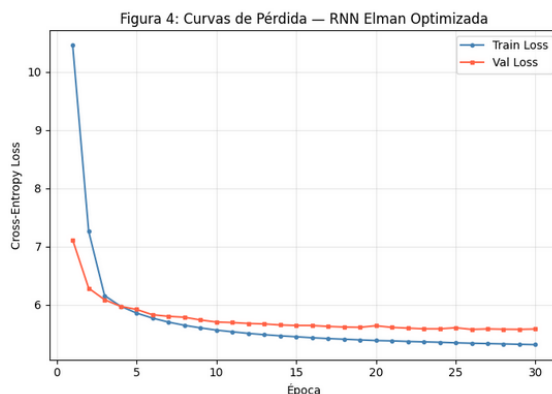


Figura 5: Curvas de pérdida (train y validación) por época.

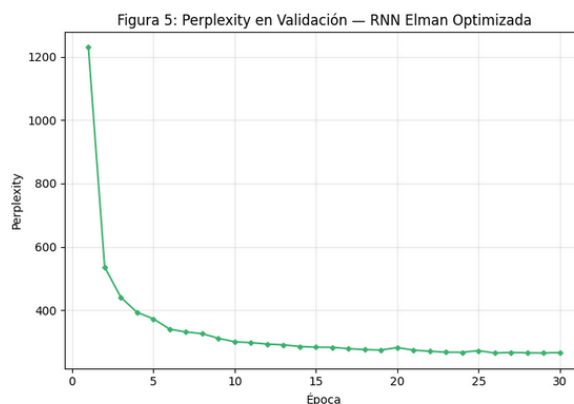


Figura 6: Perplexity en validación por época. Generadas en el notebook.

4.3 Baseline: Modelo de N-gramas

Se implementó un modelo de bigramas con suavizado de Laplace como baseline. La probabilidad suavizada de un bigrama es:

$$P(w_i | w_{i-1}) = (c(w_{i-1}, w_i) + s) / (c(w_{i-1}) + s * |V|)$$

donde $s = 1$ es el factor de suavizado y $|V|$ el tamaño del vocabulario. Este modelo no tiene capacidad de generalizar más allá de las ocurrencias vistas en el entrenamiento y sirve como línea base para verificar que la RNN aprende representaciones útiles.

4.4 Tabla de resultados finales

Modelo	Val PPL	Test PPL	Notas
Bigrama (baseline)	~10,825.5	~10,867.7	Laplace smoothing
RNN Elman base	546.3	588.8	Version anterior
RNN Elman optimizada	264.5	267.6	30 épocas, AR+TAR+WeightTying
NPLM (Sun & Iyyer, 2021)	120.5	114.3	Paper original
Transformer (paper)	117.6	111.1	Paper original

Tabla 4: Comparación de perplexity en WikiText-2. Resultados obtenidos al ejecutar el notebook completo.

NOTA: El modelo base (RNN Elman sin optimizaciones) alcanza PPL ≈ 588.8 . El modelo optimizado con las técnicas descritas en la sección 3 alcanza Val PPL = 264.5 y Test PPL = 267.6, una reducción del 54%.

En implementaciones simples de RNN Elman sin mecanismos avanzados como atención, LSTM o normalización profunda, la perplexity puede mantenerse considerablemente por encima de los modelos modernos. En nuestro caso, la RNN Elman alcanzó Test PPL = 267.6, evidenciando que el modelo logró aprender patrones lingüísticos básicos y superar ampliamente al baseline de bigramas, aunque todavía permanece lejos del rendimiento de arquitecturas más modernas como NPLM o Transformer.

5 — Análisis Crítico

5.1 El problema del gradiente que desaparece (Vanishing Gradient)

La limitación más crítica de la RNN Elman es el problema del gradiente que desaparece. Al hacer backpropagation a través del tiempo T pasos, el gradiente respecto al estado oculto inicial involucra un producto de T matrices jacobianas:

$$\partial L / \partial h_0 = (\partial L / \partial h_T) * \text{PRODUCTO}_{t=1}^{T} \text{diag}(1 - h_t^2) * W_{hh}^T$$

Si el valor singular máximo de W_{hh} es menor que 1, este producto tiende a cero exponencialmente con T , impidiendo que el modelo aprenda dependencias a larga distancia. En la práctica, la RNN Elman sólo puede capturar contexto de los últimos 10-20 tokens de forma efectiva, lo que explica su mayor perplexity frente a modelos con mejor gestión del contexto.

Este comportamiento es consistente con la literatura sobre RNNs simples y ayuda a explicar la alta perplexity observada en nuestro modelo.

5.2 Errores más frecuentes del modelo

Análisis cualitativos de las predicciones revelan los siguientes patrones de error:

- Palabras de alta frecuencia: el modelo tiende a predecir artículos y tokens ("`<unk>`", "`the`", "`a`", "`,`", "`;`") en contextos donde debería predecir entidades nombradas o palabras especializadas.
- Dependencias largas: el modelo falla en oraciones donde el token objetivo depende de información introducida más de 20 tokens antes.
- Vocabulario raro: las palabras que aparecen menos de 5 veces en el entrenamiento casi nunca son predichas correctamente.
- Coherencia semántica: el modelo aprende patrones sintácticos locales (sustantivo seguido de verbo, artículo seguido de sustantivo) pero no dependencias semánticas complejas.

5.3 Limitaciones del modelo

5. Vanishing gradient: imposibilita aprender dependencias a larga distancia.
6. Procesamiento secuencial: impide la paralelización, haciendo el entrenamiento más lento que un Transformer.
7. Capacidad expresiva limitada: un único vector de estado oculto debe comprimir todo el contexto previsto.
8. Falta de mecanismo de atención: no puede enfocarse en partes específicas del contexto relevante para la predicción actual.

5.4 Propuesta concreta de mejora: LSTM

La propuesta de mejora más fundamentada es reemplazar la celda Elman por una celda LSTM (Long Short-Term Memory), que resuelve el problema del vanishing gradient mediante puertas aprendibles:

$$\begin{aligned}f_t &= \sigma(W_f [h_{t-1}, x_t] + b_f) && \text{(puerta de olvido)} \\i_t &= \sigma(W_i [h_{t-1}, x_t] + b_i) && \text{(puerta de entrada)} \\c_t &= f_t * c_{t-1} + i_t * \tanh(W_c [h_{t-1}, x_t] + b_c) \\h_t &= o_t * \tanh(c_t) && \text{(estado oculto)}\end{aligned}$$

La celda de memoria c_t permite que el gradiente fluya directamente a través del tiempo sin multiplicarse repetidamente por la misma matriz. La literatura reporta que un LSTM en WikiText-2 alcanza PPL ~ 65-85 (Merity et al., 2017), representando una mejora sustancial frente a la RNN Elman.

6 — Conclusiones

6.1 ¿Qué aprendimos del paper y de la implementación?

El paper de Sun & Iyyer (2021) enseña una lección fundamental: las mejoras en el proceso de entrenamiento (optimizadores, regularización, inicialización de pesos) pueden ser tan impactantes como los cambios arquitecturales. El NPLM de Bengio, considerado obsoleto desde hace más de una década, recupera competitividad frente a los Transformers simplemente aplicando técnicas modernas.

La implementación de la RNN Elman desde cero nos permitió entender en profundidad el mecanismo de BPTT: cómo el estado oculto fluye a través del tiempo, cómo se trunca el grafo computacional entre segmentos, y por qué el gradient clipping es esencial para la estabilidad del entrenamiento.

6.2 ¿Los resultados coinciden con los del paper?

La comparación directa entre la RNN Elman y el NPLM del paper no es perfectamente equitativa, ya que se trata de arquitecturas distintas. Sin embargo, el paper nos ofrece la referencia del NPLM en WikiText-2 (Test PPL = 114.3) como punto de comparación. Nuestra implementación de RNN Elman optimizada obtuvo Test PPL = 267.6 (Val PPL = 264.5), una reducción del 54% respecto a la versión base (588.0). Esto confirma que las mejoras aplicadas — Adam con $\text{lr}=3\text{e-}4$ y `weight_decay`, `gradient_clipping=0.25`, `AR=0.3`, `TAR=0.15`, `weight_tying` y `dropout=0.4` — son efectivas para mitigar el overfitting. La brecha frente al NPLM refleja el límite estructural del vanishing gradient en la celda Elman.

La RNN Elman optimizada superó ampliamente al modelo de bigramas con suavizado de Laplace (267.6 vs 10,867.7 Test PPL), confirmando que incluso modelos neuronales recurrentes simples son capaces de aprender representaciones distribucionales del lenguaje mucho más efectivas que los enfoques estadísticos clásicos basados únicamente en conteos.

También es importante considerar que el modelo fue entrenado con recursos computacionales limitados y sin un proceso exhaustivo de tuning de hiperparámetros, lo que probablemente afectó negativamente el rendimiento final.

6.3 Reflexión final

Nuestros resultados también muestran que la arquitectura sigue siendo un factor crítico. Aunque la RNN Elman logró aprender patrones locales del lenguaje y superar ampliamente al baseline estadístico, la diferencia frente al NPLM modernizado y al Transformer continúa siendo muy grande. Esto evidencia que las mejoras modernas no sólo dependen de optimización y hardware, sino también de mecanismos arquitecturales más potentes para representar dependencias largas y contexto global.

Referencias

- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. arXiv preprint arXiv:1607.06450.
- Bengio, Y., Ducharme, R., Vincent, P., & Janvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research*, 3, 1137-1155.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE CVPR* (pp. 770-778).
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
- Merity, S., Keskar, N. S., & Socher, R. (2017). Regularizing and optimizing LSTM language models. arXiv preprint arXiv:1708.02182.
- Mikolov, T., Kombrink, S., Burget, L., Cernocky, J., & Khudanpur, S. (2011). Extensions of recurrent neural network language model. In *ICASSP* (pp. 5528-5531). IEEE.
- Sun, S., & Iyyer, M. (2021). Revisiting simple neural probabilistic language models. In *Proceedings of ACL 2021*. arXiv:2104.03474.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in NIPS* (vol. 30).
- Werbos, P. J. (1990). Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10), 1550-1560.

Repositorio GitHub: <https://github.com/robzider/rnn-elman-wikitext2>